

Paradigmas de Programación

TRABAJO PRÁCTICO N° 3

1. Ejercicios

Ejercicio 1.1. Para poder desarrollar software en forma eficiente existen diferentes paradigmas de programación que se pueden adaptar a las necesidades del proyecto y su dominio de aplicación. Crear una tabla que vincule los dominios de aplicación con los diferentes paradigmas y lenguajes de programación utilizados.

Ejercicio 1.2. El concepto de abstracción es un pilar de la POO, pero no así de los lenguajes puramente procedimentales. Explicar por qué, si esto es realmente verdad.

Ejercicio 1.3. El concepto de abstracción es también ampliamente aplicado en los lenguajes declarativos. Explicar por qué.

Ejercicio 1.4. El paradigma declarativo se enfoca en qué debe computarse, en vez de cómo debe computarse. VERDADERO o FALSO. Justificar su respuesta.

Ejercicio 1.5. En los lenguajes declarativos la lógica algorítmica o de control de flujo se maneja internamente por el sistema. VERDADERO o FALSO. Justificar su respuesta.

Ejercicio 1.6. De los siguientes lenguajes de programación: C++, C#, Java, JavaScript y Python (todos orientados a objetos), explicar y consignar en una tabla sus resultados sobre los siguientes aspectos:

- Abstracción
- Encapsulamiento
- Herencia (simple o múltiple)
- Polimorfismo (cómo se implementa y tipos de polimorfismo)

Ejercicio 1.7. Indicar la respuesta correcta (solo hay una respuesta correcta): Imperativo, declarativo y orientado a objetos son:

- a) Modos de compilar el código fuente de un programa de ordenador.
- b) Modos de definir el pseudocódigo de un programa de ordenador.
- c) Paradigmas de programación.
- d) Las anteriores respuestas no son correctas.

Ejercicio 1.8. Python, C++ y otros lenguajes orientados a objetos permiten la herencia múltiple basada en clases, lo cual puede generar ambigüedades. VERDADERO o FALSO. Justificar su respuesta con un ejemplo concreto de alguno de los lenguajes listados.

Ejercicio 1.9. Los lenguajes de programación pueden implementarse mediante tres métodos generales: compilación, interpretación e híbrido. Los programas que pueden traducirse a lenguaje máquina de modo que pueden ejecutarse directamente en la computadora aplican el método de implementación llamado _____. Otro enfoque en el extremo opuesto actúa como una simulación de software

procesando sentencias de programa de lenguaje de alto nivel en lugar de instrucciones de máquina. Esta simulación de software, obviamente, proporciona una máquina virtual para el lenguaje y se corresponde con el modelo llamado _____. Otras implementaciones buscan un equilibrio, traduciendo programas de lenguaje de alto nivel a un lenguaje intermedio diseñado para facilitar su interpretación, y denominarse sistemas de implementación _____.

Ejercicio 1.10. Explicar cuáles son las similitudes, diferencias, ventajas y desventajas entre la compilación, interpretación y los sistemas híbridos.

Ejercicio 1.11. Construir una tabla de símbolos básica (un solo ámbito). Dado el siguiente fragmento en C, construir la tabla de símbolos del ámbito global y del cuerpo de `main`. Para cada identificador, incluir al menos: **lexema**, **clase** (variable/función), **tipo**, **ámbito**, **dirección/offset** (si aplica) y **tiempo de vida** (estático/automático). **Indicar** cuáles entradas se crean en tiempo de compilación y cuáles se determinan finalmente en enlace/carga.

```
1 int g = 10;
2
3 int suma(int a, int b) {
4     int t = a + b;
5     return t;
6 }
7
8 int main(void) {
9     int x = 5;
10    int y = suma(x, g);
11    return y;
12 }
```

Ejercicio 1.12. Indicar cuál de las siguientes afirmaciones es cierta y **justificar** la respuesta:

- a) Un analizador léxico o *scanner* es un programa que divide una secuencia de caracteres (el programa) en una secuencia de componentes sintácticos primitivos: las instrucciones.
- b) Un analizador léxico es un programa que divide una secuencia de caracteres (el programa) en una secuencia de componentes semánticos: las instrucciones.

Ejercicio 1.13. Teniendo en cuenta las fases del proceso de compilación, indicar (basándose en la figura provista) cómo se transmite la información entre ellas y redactar a continuación un pequeño detalle de cuál es el fin de cada fase del proceso.

Ejercicio 1.14. Escribir una función semántica simple que evalúe el significado de una expresión aritmética representada como un árbol (AST). Ejemplo: para $3 + (2 * 4)$ el árbol debe producir el valor 11.

Ejercicio 1.15. Explicar para qué sirve la **Tabla de Símbolos**. ¿Se tiene en cuenta en todo el proceso? Justificar.

Ejercicio 1.16. Indicar cuál de las siguientes afirmaciones es cierta y **justificar** la respuesta:

- a) La compatibilidad de tipos, el alcance de las variables y la signatura de las funciones (coincidencia del número de argumentos formales en una llamada) forman parte de la **semántica estática** del lenguaje.
- b) La compatibilidad de tipos, el alcance de las variables y la signatura de las funciones (coincidencia del número de argumentos formales en una llamada) forman parte de la **sintaxis** del lenguaje.

Ejercicio 1.17. Clasificar los siguientes lenguajes de programación indicando si son **compilados**, **interpretados** o **híbridos**: C, C++, COBOL, Fortran, Haskell, Java, Pascal, Perl, PHP, Python y Scala.

Ejercicio 1.18. Explicar las características de una variable vistas en clases.

Ejercicio 1.19. Indicar la respuesta correcta y justificar:

- a) Un tipo describe un conjunto de valores posibles a asignar a una expresión.

- b) Un tipo describe un conjunto de operaciones predefinidas.
- c) Un tipo describe un conjunto de valores y operaciones predefinidos para esos valores.

Ejercicio 1.20. Definir con ejemplos la diferencia entre sintaxis y semántica. Dar un ejemplo de un programa que sea sintácticamente correcto pero semánticamente incorrecto

Ejercicio 1.21. Explicar qué es el tipado estático y listar al menos tres lenguajes representativos.

Ejercicio 1.22. Explicar qué es el tipado dinámico y listar al menos tres lenguajes representativos.

Ejercicio 1.23. Explicar qué tipo de tipado tienen Java, Python, Prolog y Haskell.

Ejercicio 1.24. ¿Cuándo un lenguaje tiene un tipado fuerte? Explicar y ejemplificar. Incluir al menos un ejemplo con `lstlisting`.

Ejercicio 1.25. En un lenguaje con tipado fuerte no se puede operar entre distintos tipos en forma implícita, aunque sí en forma explícita. Explicar y ejemplificar.

Ejercicio 1.26. ¿El concepto de ligadura está relacionado con la sintaxis o con la semántica de un lenguaje de programación? Justificar y ejemplificar su respuesta.

Ejercicio 1.27. Una ligadura es una asociación entre un atributo y una entidad, como entre una variable y su tipo o valor, o entre una operación y un símbolo. ¿Cuál de las siguientes afirmaciones es **falsa**? Justificar.

- a) La ligadura y el tiempo de ligadura son conceptos importantes en la semántica de los lenguajes de programación.
- b) La ligadura y el tiempo de ligadura son conceptos importantes en la sintaxis de los lenguajes de programación.
- c) El momento en que se produce la ligadura o enlace se denomina tiempo de ligadura.
- d) Las ligaduras pueden tener lugar en el diseño, la implementación, la compilación, la carga, el enlace o la ejecución del lenguaje.

Ejercicio 1.28. La ligadura puede ser tanto estática como dinámica. ¿De qué depende que sea de un tipo u otro? Justificar.

Ejercicio 1.29. La ligadura de nombre de una variable es estática y se da en tiempo de implementación. VERDADERO o FALSO. Justificar.

Ejercicio 1.30. La ligadura de tipo de una variable en Java es estática mientras que en Python es dinámica. VERDADERO o FALSO. Justificar.

Ejercicio 1.31. ¿Cuál es la diferencia entre una ligadura explícita y una ligadura implícita? Justificar y ejemplificar su respuesta.

Ejercicio 1.32. Explicar qué diferencia a un lenguaje de dominio de un lenguaje de propósito general.

Ejercicio 1.33. Explicar cuáles son las características de un lenguaje de dominio o lenguaje específico de dominio.

Ejercicio 1.34. Explicar cuáles son las características de un lenguaje de *scripting*.

Ejercicio 1.35. Explicar cuáles son las características de un lenguaje de *pegamento*.

Ejercicio 1.36. Los lenguajes de dominio (o lenguajes específicos de dominio) pueden ser internos (embebidos) o externos. Por ejemplo, para Haskell tenemos QuickCheck como DSL embebido (<https://hackage.haskell.org/package/QuickCheck>) y en el caso de HTML utilizado en la web se considera un DSL externo. Sin embargo, hace uso de lenguajes embebidos, como JavaScript del lado del cliente (navegador).

Investigar y proporcionar **tres** ejemplos de lenguajes DSL tanto internos como externos. Justificar.

Ejercicio 1.37. ¿Los lenguajes de *scripting* son interpretados o compilados? Justificar su respuesta.

Ejercicio 1.38. En general los lenguajes de *scripting* están enfocados en administración de sistemas, web y tareas de automatización. Por ejemplo, rspamd es un sistema de filtrado de spam de código abierto (<https://rspamd.com/>) y utiliza el lenguaje Lua como lenguaje de scripting para configurar reglas.

Investigar y proporcionar **dos** ejemplos con otros lenguajes de scripting. Justificar.

Ejercicio 1.39. ¿Por qué los lenguajes de *scripting* también suelen ser lenguajes de *pegamento*?

Ejercicio 1.40. Para reflexionar: ¿por qué los lenguajes de scripting suelen ser más flexibles pero menos eficientes que los compilados? ¿Sucede esto en realidad o es solo ficción inventada por los haters de un determinado lenguaje?. Justificar su respuesta. Escribe ejemplos de decisiones de diseño en Python que favorecen la simplicidad (ejemplo: tipado dinámico).

Ejercicio 1.41. Implementar un mini-intérprete en Python que reciba como entrada operaciones aritméticas básicas en forma de string ("2+3*4") y muestre el resultado.